CS 595 Lab Notebook

Lab 1

Name: Jarren Calizo
OdinID: calizo
Date: April 12, 2025

Table of Contents

1. *Lab Overview* 2

2. *Lab 1.2*

    - Setup and Sequential Fetcher Program 3
    - Measuring Execution Time and Multiprocessing Performance 3
    - Downloading URL Sets and Visualizing Multiprocessing Speedup 5
    - Implementing Asynchronous Fetching with asyncio + httpx 6
    - Final Asynchronous Graph and Comparison 7

3. *Lab 1.3*

    - Inspecting Authentication POST Request 8
    - Username and Password Enumeration via HTTP Response Analysis 9
    - Broken Brute-Force Protection, IP Block 9
    - Username Enumeration via Account Lock: Credential Discovery and Exploitation 10
    - OAuth Flow Analysis: Authorization Parameters and Redirection 12
    - OAuth Flow: Authentication, Consent, and Access Token Retrieval 13
    - Identity Injection via Client-Side Token Handling 14
    - Automating the OAuth Authorization Flow via Python 15
    - OAuth Callback Analysis and Implicit Flow 17
    - OAuth Exploitation and Final Account Takeover 19

Lab Overview

The first set of labs introduced critical real-world vulnerabilities in authentication systems and the infrastructure used to deploy and test them. Starting with foundational setup in Lab 1.1 and progressively diving into logic flaws, timing attacks, and OAuth misconfigurations, these labs emphasized the mindset and methodology behind offensive security testing. I began by creating course-related accounts: PortSwigger for labs, GitLab for version control, and Slack for collaboration. I provisioned Google Cloud Platform (GCP) resources, deployed two WebForPentester VMs (WFP1 and WFP2), and configured strict firewall rules. Using gcloud, I locked access to only my Personal IPs and Portland State ranges (131.252.0.0/16) and tagged the VMs with pdx-80 and pdx-22 for HTTP/SSH. This hardened my testing environment while conserving resources by stopping unused VMs between sessions.

I wrote Python programs to fetch webpage titles, first sequentially using requests, then in parallel using multiprocessing.Pool. To analyze performance scaling, I fetched from 10–30 URLs at varying process counts and visualized concurrency vs. runtime using matplotlib. I followed this with an asyncio + httpx version, measuring how asynchronous I/O compares to CPU-bound multiprocessing. The lab reinforced the trade-offs in system design between simplicity, resource use, and execution speed.

This lab walked through how seemingly minor login response differences can leak information. Using requests and BeautifulSoup, I first bypassed username enumeration by checking server error messages. In the second part, I defeated IP-based brute-force protection by rotating between valid and invalid logins (wiener:peter used as a decoy). I successfully brute-forced the carlos account. Finally, I exploited lockout behavior to discover usernames (e.g., as400) and brute-forced the password (zxcvbnm) after sleeping out the cooldown.

Lab 1.4 emphasized more covert login enumeration. Rather than using response body differences or error strings, I measured server response time via requests.elapsed.total_seconds() for a fixed password (64 'a's) across a list of usernames. Valid usernames incur longer processing time as the server likely performs deeper checks (e.g., password hash comparison). I iterated each test using spoofed IPs and multiple trials (with statistics.mean) to smooth noise. Once the top N usernames were ranked by timing, I brute-forced each with the password list. The key takeaway was tuning for reliability—balancing trial count, candidate breadth, and runtime performance.

Across Labs 1.1–1.4, I honed my offensive security mindset—automating attacks, reading subtle signals like timing and error responses, and tuning scripts for reliability. Each lab pushed me to think like an attacker while executing like an engineer, reinforcing both technical depth and strategic awareness.

Lab 1.2 Web Programming - Parallelism and Performance

Setup and Sequential Fetcher Program

To begin this lab, I initialized a new Python project for CS 595 under the directory:

- ~/OneDrive/Documents/GitProjects/School/websec-calizo-calizo/labs/lab1.2

Following instructions from Step 1, I created a virtual environment using the venv module and activated it within Git Bash. After verifying the Python path was available through Visual Studio Code, I installed the requests and beautifulsoup4 packages locally. The notebook directory was structured with the following contents:

- lab1.2/ # lab source code
- notebooks/Labs1.pdf # PDF upload target

I implemented the getUrlTitle() and getSequential() functions, which used the requests package to download HTML content and BeautifulSoup to parse the <title> tag from each page.

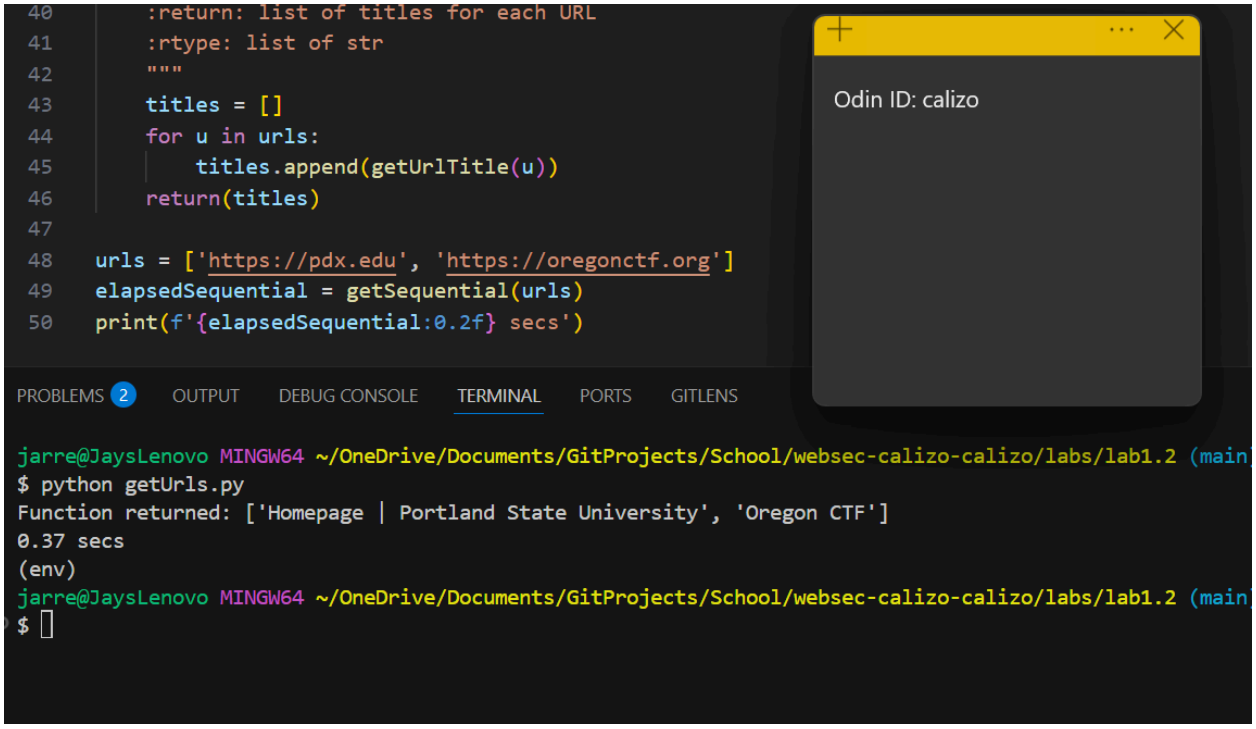
When tested against ['https://pdx.edu', 'https://oregonctf.org'], the program returned the expected output:

- ['Homepage | Portland State University', 'Oregon CTF']

Measuring Execution Time and Multiprocessing Performance

I created a @time_decorator wrapper to instrument and measure the runtime of getSequential(). This function prints the result of the function call and returns the elapsed time.

The updated version printed both the page titles and the total runtime for sequential processing.

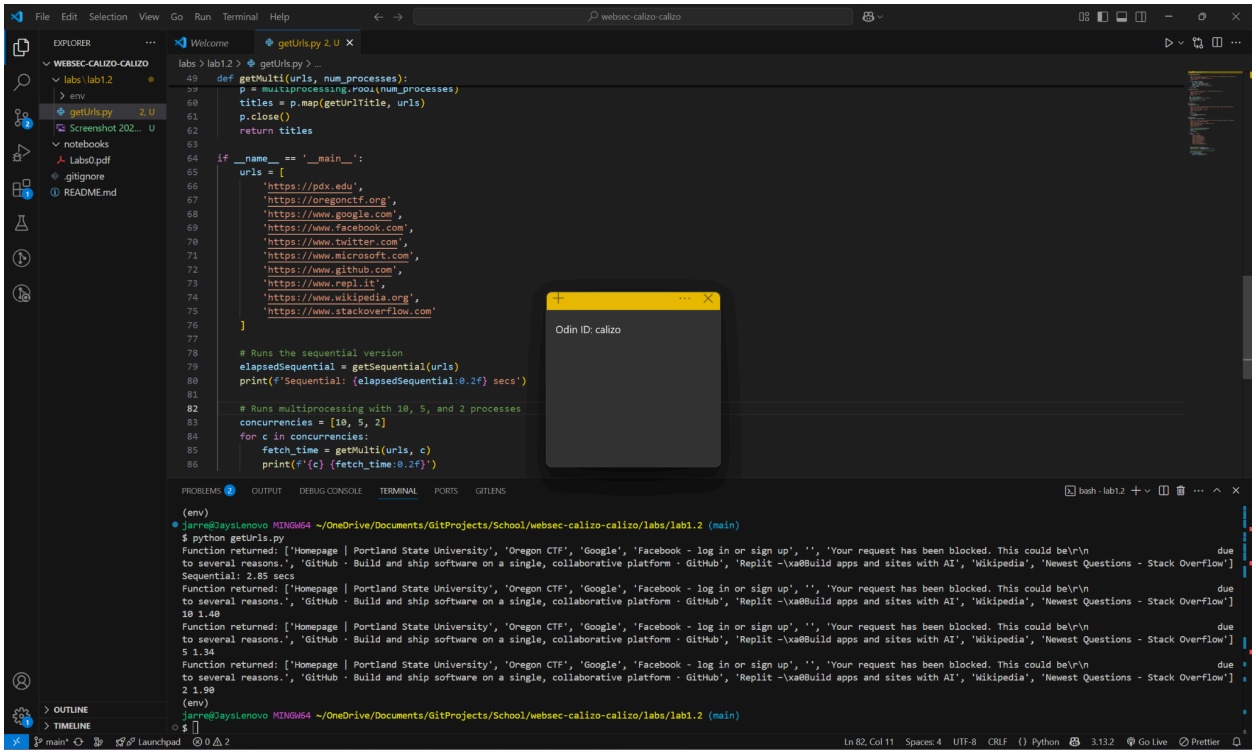


This shows successful terminal output with a runtime of approximately 0.37 seconds.

To explore performance improvements via multiprocessing, I implemented the getMulti() function using multiprocessing.Pool. This function distributes URLs among num_processes workers and gathers titles in parallel.

A list of 10+ popular URLs was used:
- urls = [
    'https://pdx.edu', 'https://oregonctf.org', 'https://www.google.com',
    'https://www.facebook.com', 'https://replit.com', 'https://www.microsoft.com',
    'https://www.github.com', 'https://www.pep1.it', 'https://www.wikipedia.org',
    'https://www.stackoverflow.com']

Using a for loop, I invoked getMulti() with varying process counts. Results showed that higher parallelism (10 or 5 processes) improved performance over 2 processes, though not always linearly due to system limitations.



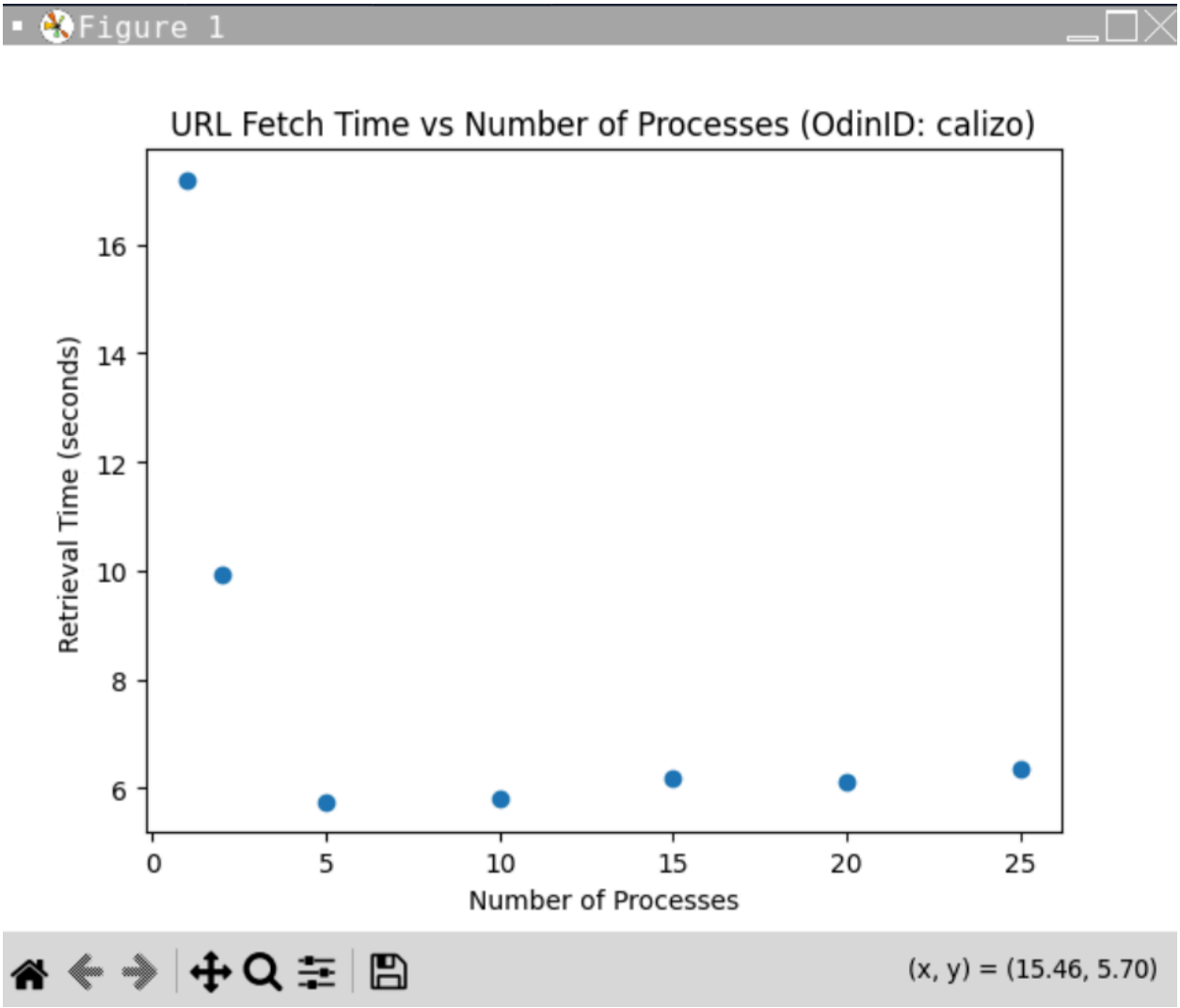
This shows the printed output with function returns. Even when some URLs failed to resolve, runtime decreased as concurrency increased.

Downloading URL Sets and Visualizing Multiprocessing Speedup

For the final performance visualization, I retrieved a longer list of URLs using:

- resp = requests.get('https://thefengs.com/wuchang/courses/cs495/urls.txt')
- urls = resp.text.strip().split('\n')

I then tested getMulti() across concurrencies [25, 20, 15, 10, 5, 2, 1] and stored the elapsed times. Using matplotlib.pyplot, I plotted a scatter plot showing concurrency level vs. total runtime.



This clearly shows a performance plateau after ~10 processes.

Implementing Asynchronous Fetching with asyncio + httpx

To reduce overhead compared to multiprocessing, I implemented agetUrlTitle() as an asynchronous coroutine using httpx.AsyncClient. This yielded control during each network request, improving efficiency.

I implemented async_main() to asynchronously schedule and await all URL fetches. The synchronous wrapper getAsync() used asyncio.run() to trigger this inside a timed function.

- @time_decorator
  def getAsync(urls):
    return asyncio.run(async_main(urls))

To compare fairly with the multiprocessing model, I batched the async fetches using slice ranges:

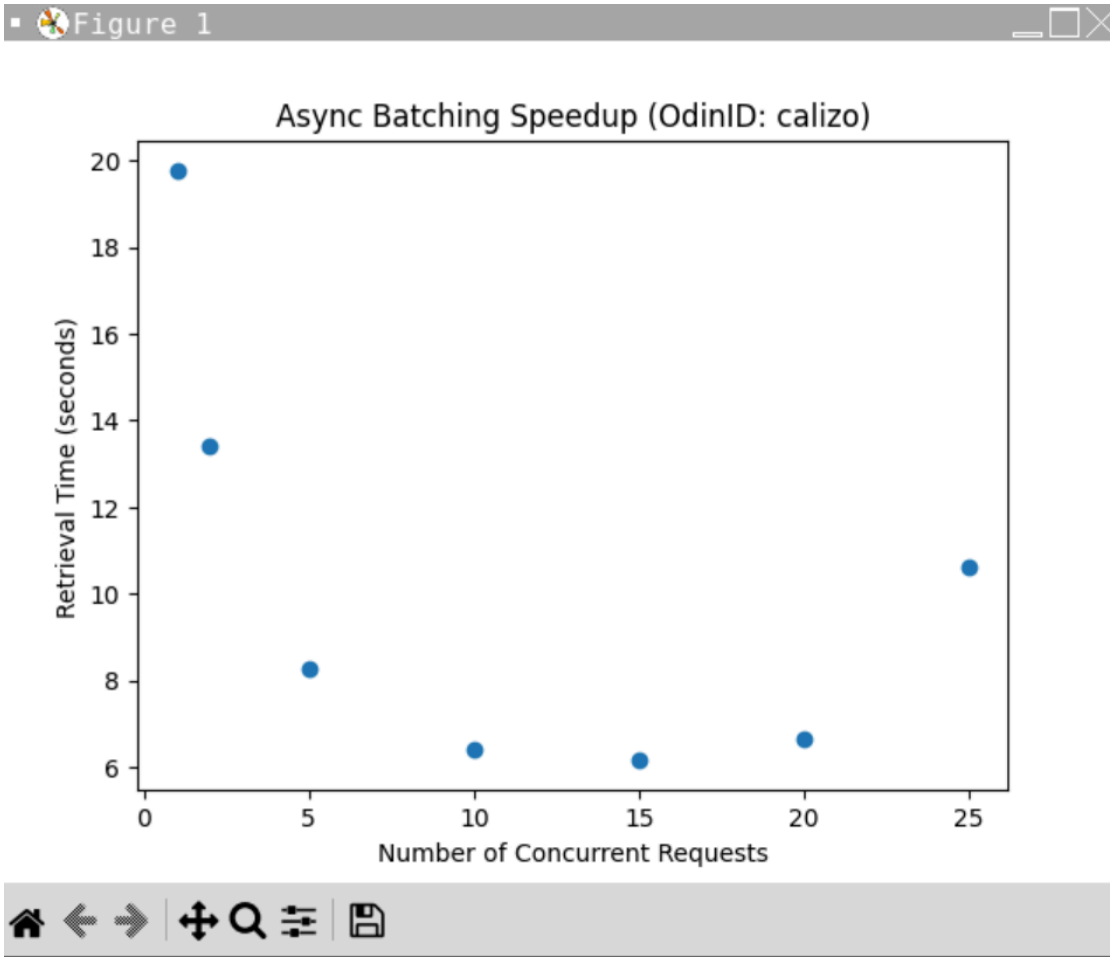
- for i in range(0, len(urls), concurrency):
  fetch_time = getAsync(urls[i:i+concurrency])

This limited concurrent I/O per batch while avoiding OS saturation.

Elapsed times for each concurrency were appended and printed:
[(25, 10.606292637996376), (20, 6.660707301991351), (15, 6.149630204992718), (10, 6.388450090009428), (5, 8.283135059013148), (2, 13.419460748002166), (1, 19.75604010897223)]

Final Asynchronous Graph and Comparison

The final plot illustrated async batching runtimes across varying concurrency levels.



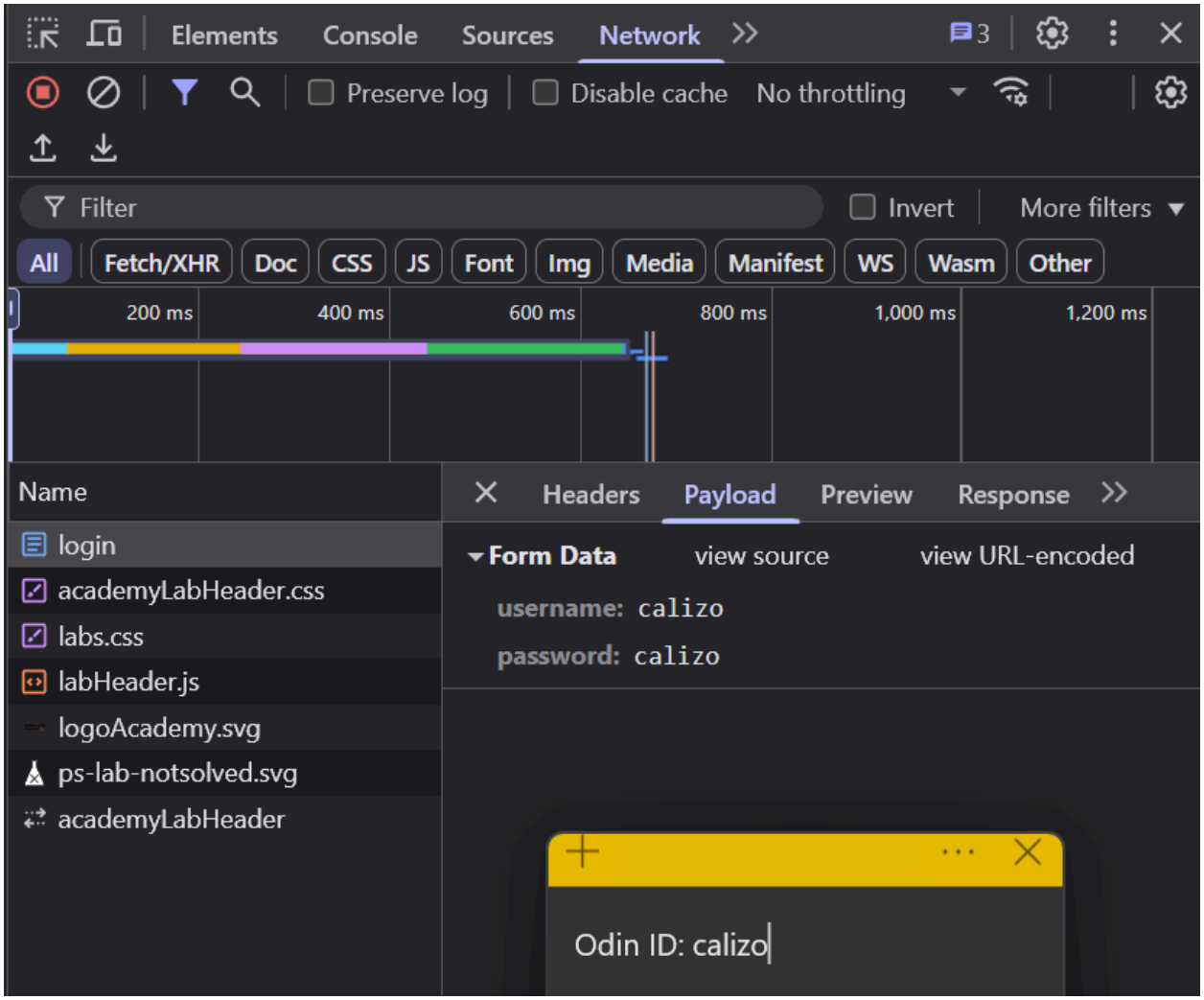
This completes Lab 1.2. The results show clear advantages of parallelization—both via multiprocessing and async—but also highlight tradeoffs in system complexity and performance behavior across different models.

Lab 1.3 Broken Authentication - Exploiting Login Logic and Credential Flaws

Inspecting Authentication POST Request

In this step, I accessed the authentication lab focused on username enumeration and navigated to the login form. Using Brave's Developer Tools, I inspected the HTML structure of the form and verified that it performs a POST request to /login with the parameters username and password.

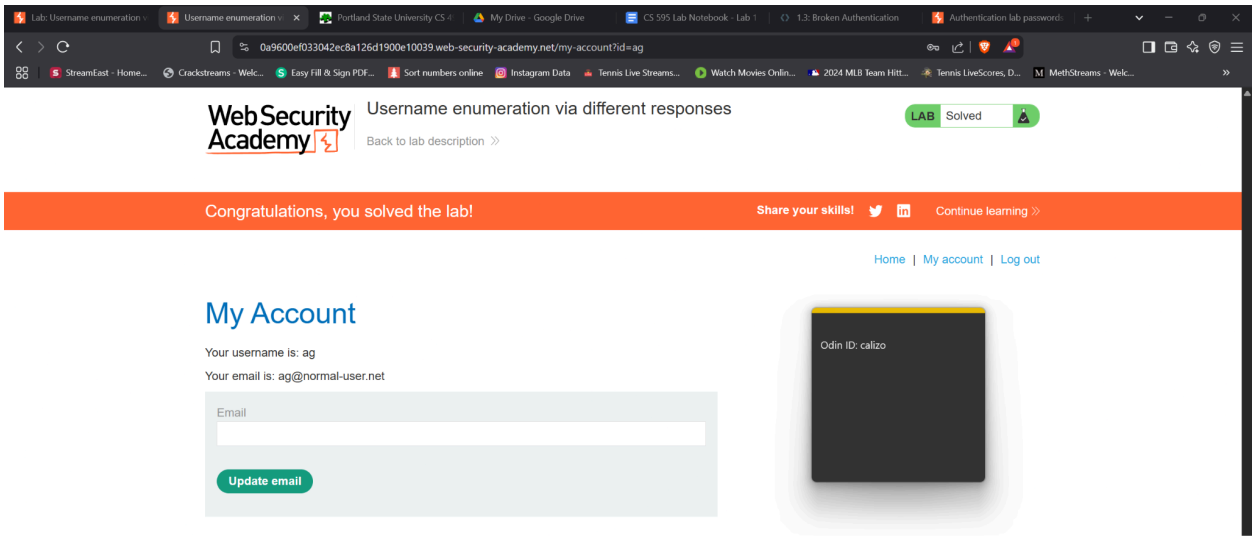
After submitting my OdinID as both the username and password, I reviewed the Network tab and confirmed the form's submission as an HTTP POST. The request headers and payload were both visible, confirming that the browser sent my input in the body of the request. This screenshot shows the payload for the login attempt:



Username and Password Enumeration via HTTP Response Analysis

In this lab, I exploited response differences in the login workflow to first enumerate a valid username (ag) and then brute-force the password (sunshine) using Python and the requests/BeautifulSoup packages.

My first script checked for the absence of "Invalid username" in the server's response, identifying a valid account. The second script leveraged HTTP status codes — specifically a 302 Found redirect — to determine the correct password. I handled edge cases such as timeouts and ensured robust input processing using local text files (auth-lab-usernames.txt, auth-lab-passwords.txt).



The successful login was verified by accessing the authenticated user's profile page and capturing a screenshot of the completion message, URL, and my OdinID.

Broken Brute-Force Protection, IP Block

In this lab, I bypassed a web application's IP-based brute-force protection to compromise the account of a known user, carlos. The server implemented a basic defense mechanism where repeated failed login attempts from a single IP would trigger a temporary lockout. However, this mechanism could be trivially reset by successfully logging into another account from the same IP address.

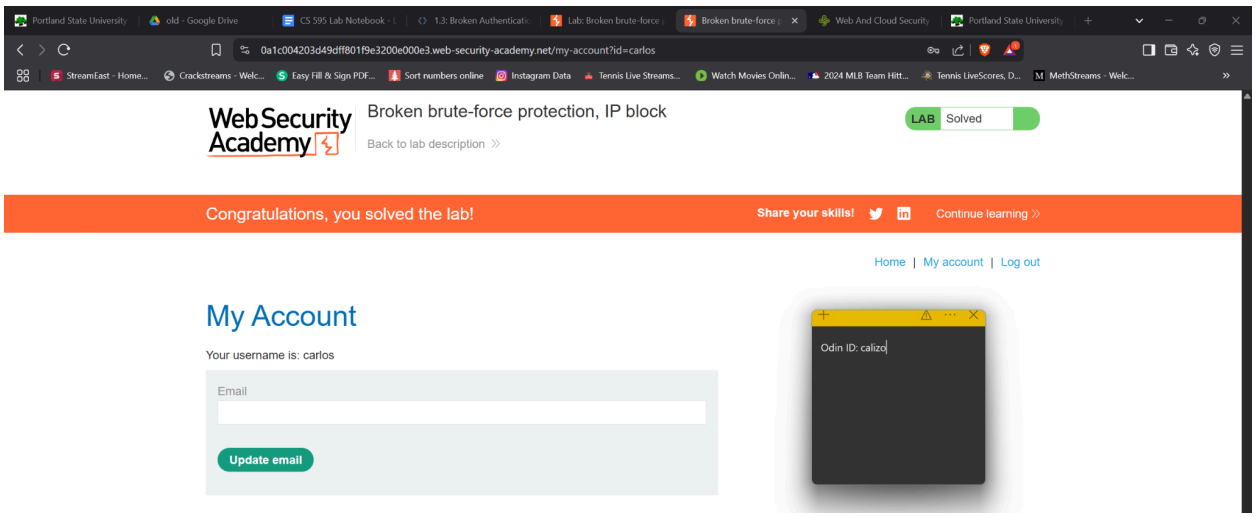
The application supplied legitimate credentials for the user wiener (wiener:peter), which I used as a decoy to reset the lockout counter after every failed login attempt for carlos.

I created a Python script that:

- Iterates through a supplied password list (auth-lab-passwords.txt)
- For each password attempt, tries logging in as carlos

- If login fails, it immediately logs in as wiener to reset the lockout
- Detects a successful login by checking for an HTTP 302 Found redirect

This strategy allowed me to brute-force the correct password (password) for carlos without being blocked by IP lockout controls. The lab was successfully completed and verified with a screenshot of the profile page containing the carlos account and my OdinID:



Username Enumeration via Account Lock: Credential Discovery and Exploitation

In this portion of the lab, we were challenged to exploit a subtle logic flaw in the web application’s authentication mechanism—specifically one involving account lockouts. The vulnerability emerges when a valid username is submitted multiple times with an invalid password. After 5 incorrect attempts, the server reveals a unique message: "You have made too many incorrect login attempts". This differing behavior allows for username enumeration, even though submitting invalid usernames always returns the generic message: "Invalid username or password".

To take advantage of this flaw, I implemented a script (account_lock_enum.py) that does the following:

1. **Lockout-Based Username Enumeration**

- def try_target(username):

```
...
for i in range(6):
    resp = s.post(login_url, data=logindata)
return resp.text
```

- I wrote a helper function try_target() that attempts to log in as a given username six times in a row using the bogus password 'foo'. After exhausting those attempts, I check whether the server responds with the lockout message. If so, we’ve positively identified a valid user account. In this case, the username as400 triggered the lockout behavior and was identified as valid.

2. Password Enumeration with Conditional Logic

- for pw in passwords:

  …
  if 'incorrect login attempts' not in resp.text and 'Invalid username or password' not in resp.text:

- Once the valid username was known, I parsed through a common password list to brute-force the correct password. The application displays either a lockout message or a generic invalid login message for wrong passwords—but remains silent when the correct password is provided. This “quiet success” pattern enabled me to infer that the correct password was "zxcvbnm".

3. Handling Lockout Reset Gracefully

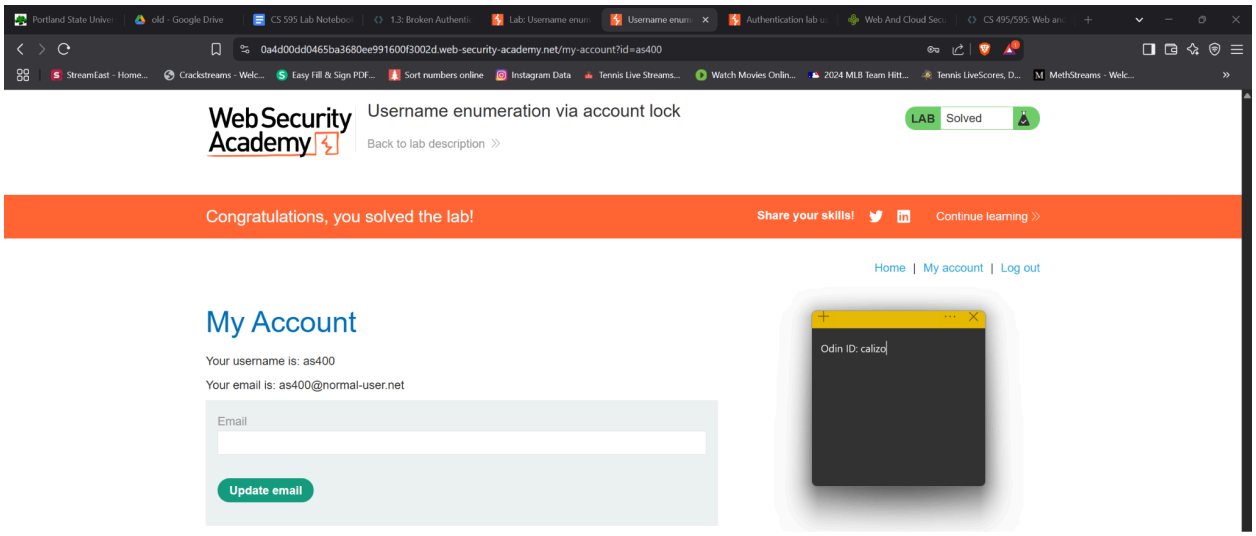
- print("\n[*] Waiting 60 seconds for account lock to reset...")
  time.sleep(60)

- Because of the earlier lockout during enumeration, I implemented a time.sleep(60) to allow the lockout timer to expire before reattempting the successful login credentials. Without this, the legitimate login would still be denied due to the imposed cooldown period.

4. Final Login Attempt

- resp = s.post(login_url, data={...})
  if resp.status_code == 302 or 'My account' in resp.text:
  ...

- With both the valid username and password, and after waiting out the lockout, the final POST request successfully authenticated and redirected to /my-account, completing the lab.



- Username discovered: as400
- Password discovered: zxcvbnm
- Lab status: Successfully completed with full understanding of lockout-based enumeration and credential rotation logic

This lab was a clear demonstration of how even well-meaning authentication mechanisms like lockouts can become side-channel signals for attackers when implemented naively. It also reinforced the value of strategic scripting and timing when developing brute-force automation.

OAuth Flow Analysis: Authorization Parameters and Redirection

To analyze the OAuth 2.0 implicit flow implemented by the client application, I used Chrome Developer Tools with “Preserve log” enabled and initiated the login process by clicking the "My Account" link. This triggered a series of redirections, ultimately forwarding the browser to the third-party OAuth identity provider.

By capturing the full request to the /auth endpoint on the identity provider’s domain, I extracted the key authorization parameters below:

- Hostname (Identity Provider):
  - oauth-0afb003b03a2a15880251565023c0038.oauth-server.net

- client_id:
  - lvaezruco8gl48ncguwgg

- redirect_uri (Callback URL):

    - https://0ad700fc038ea101804f17fe00ea0062.web-security-academy.net/oauth-callback

- scope:
    - openid profile email

(This indicates the client is requesting basic identity, profile information, and email address from the identity provider.)

- response_type:
    - token

(This confirms the use of the OAuth 2.0 Implicit Flow, in which the access token is returned directly in the URL fragment.)

- Initial Redirect to Login Form (Location header):
    - /interaction/yVsyQStaQBEvEhJ8QIywI

These values help trace how the authorization process proceeds from the client application to the identity provider and back, and will be critical in the next steps where I investigate the authentication form, consent approval, and final redirection containing the access token.

OAuth Flow: Authentication, Consent, and Access Token Retrieval

After being redirected to the identity provider's login portal, I tracked the entire OAuth flow using Chrome Developer Tools and confirmed each redirection step, form submission, and final authorization token delivery.

Upon arriving at the OAuth login screen, I inspected the login form and found the following action attribute:

- <form autocomplete="off" action="/interaction/yVsyQStaQBEvEhJ8QIywI/login" method="post">
- Login Form URL:
    - **https://oauth-0afb003b03a2a15880251565023c0038.oauth-server.net/interaction/yVsyQStaQBEvEhJ8QIywI/login**
        - This is the URL where the identity provider receives user credentials.

Using the credentials provided (wiener:peter), I submitted the form and verified that the POST request payload included:

- username=wiener
- password=peter

This submission led to a redirect to the consent page, confirming successful authentication with the identity provider.

On the consent screen, I inspected the form responsible for granting access to the client application. The form had the following markup:

- <form autocomplete="off"
  action="/interaction/yVsyQStaQBEvEhJ8QIywI/confirm" method="post">
- Consent Form Submission URL:
  - **https://oauth-0afb003b03a2a15880251565023c0038.oauth-server.net/interaction/yVsyQStaQBEvEhJ8QIywI/confirm**

Clicking "Continue" submitted this POST request to the identity provider, finalizing the OAuth authorization.

After the consent step, the browser was redirected to the original client application via the redirect_uri with an access token attached in the URL fragment:

- https://0ad700fc038ea101804f17fe00ea0062.web-security-academy.net/oauth-callback#
  access_token=xMHbxct4q384xjhkDotkM15HmELpxUhDtIguHlnM98h
  &expires_in=3600
  &token_type=Bearer
  &scope=openid%20profile%20email
- Access Token:
  - **xMHbxct4q384xjhkDotkM15HmELpxUhDtIguHlnM98h**

Identity Injection via Client-Side Token Handling

In this step, I analyzed how the client application completes the OAuth 2.0 implicit flow and discovered a key flaw in how it associates an authenticated user's identity with an internal account. Rather than validating the access token server-side, the client relies on the user's browser to self-report their identity, introducing a critical opportunity for privilege escalation.

After logging in via OAuth as the user wiener, the browser sent the previously obtained access_token to the identity provider's /me endpoint. I captured the second (non-CORS preflight) request in the Network tab:

- GET /me HTTP/1.1
  Host: oauth-0afb003b03a2a15880251565023c0038.oauth-server.net
  Authorization: Bearer xMHbxct4q384xjhkDotkM15HmELpxUhDtIguHlnM98h

The identity provider returned the following JSON payload:

{

"email": "**wiener@hotdog.com**"
}

Shortly afterward, the browser sent a POST request to the client app’s /authenticate endpoint with the following payload:

{
"email": "wiener@hotdog.com",
"username": "wiener",
"token": "xMHbxct4q384xjhkDotkM15HmELpxUhDtIguHlnM98h"
}

The fundamental issue here lies in the client application's decision to trust user-submitted identity information. Rather than validating the access_token by contacting the identity provider’s /me endpoint server-side, it delegates this verification to the user's browser.

This trust model allows a malicious user to forge any identity simply by modifying the POST request to /authenticate — replacing their email with another user's email (e.g., carlos@carlos-montoya.net) while still using a valid access token for a different user.

By taking the valid token from wiener and modifying the /authenticate request as follows:

{
"email": "carlos@carlos-montoya.net",
"username": "wiener",
"token": "xMHbxct4q384xjhkDotkM15HmELpxUhDtIguHlnM98h"
}

I will impersonate ‘carlos’ in the client application — even though I authenticated only as ‘wiener’ with the identity provider. The server’s trust in client-side identity propagation makes the attack not only trivial to execute, but fully silent from the target user’s perspective.

Automating the OAuth Authorization Flow via Python

This step involves using a Python script to automate the OAuth implicit flow, mimicking what the browser did manually in Steps 6–8. By doing this programmatically, I can capture the entire redirect chain, retrieve a valid access token, and prepare for identity forgery in the next step.

The first part of the script begins by importing necessary libraries:

```
import requests
from bs4 import BeautifulSoup
import re
```

- requests handles HTTP requests and session management.
- BeautifulSoup parses HTML content to extract forms and meta tags.
- re (regular expressions) will be used later to extract the access token from a redirect URI.

We begin the OAuth flow by initiating a session and requesting the social login page from the vulnerable client application:

```
site = '<FMI>'  # Replace with the lab’s hostname, mines being '0ad700fc038ea101804f17fe00ea0062.web-security-academy.net'

s = requests.Session()
login_url = f'https://{site}/social-login'
resp = s.get(login_url)
soup = BeautifulSoup(resp.text,'html.parser')
meta = soup.find('meta', {'http-equiv':'refresh'})
print(f'Meta tag is: {meta}')
```

- This returns a <meta http-equiv="refresh"> tag that instructs the browser to redirect after a 3-second delay.
- The script parses this tag and extracts the authorization URL:
    - auth_url = meta['content'].split(';')[1].lstrip('url=')
      print(f'Authorization URL is: {auth_url}')

- It also isolates the OAuth identity provider hostname from the URL:
    - oauth_site = auth_url.split('/')[2]
      print(f'Identity provider site is: {oauth_site}')

Next, the script replicates the login process at the identity provider:

```
resp = s.get(auth_url)
soup = BeautifulSoup(resp.text,'html.parser')
login = soup.find('form')
login_url = f"https://{oauth_site}{login['action']}"
print(f'Sign-in URL is: {login_url}')
```

It submits valid credentials (wiener:peter) to the OAuth login endpoint:

```
login_data = {
```

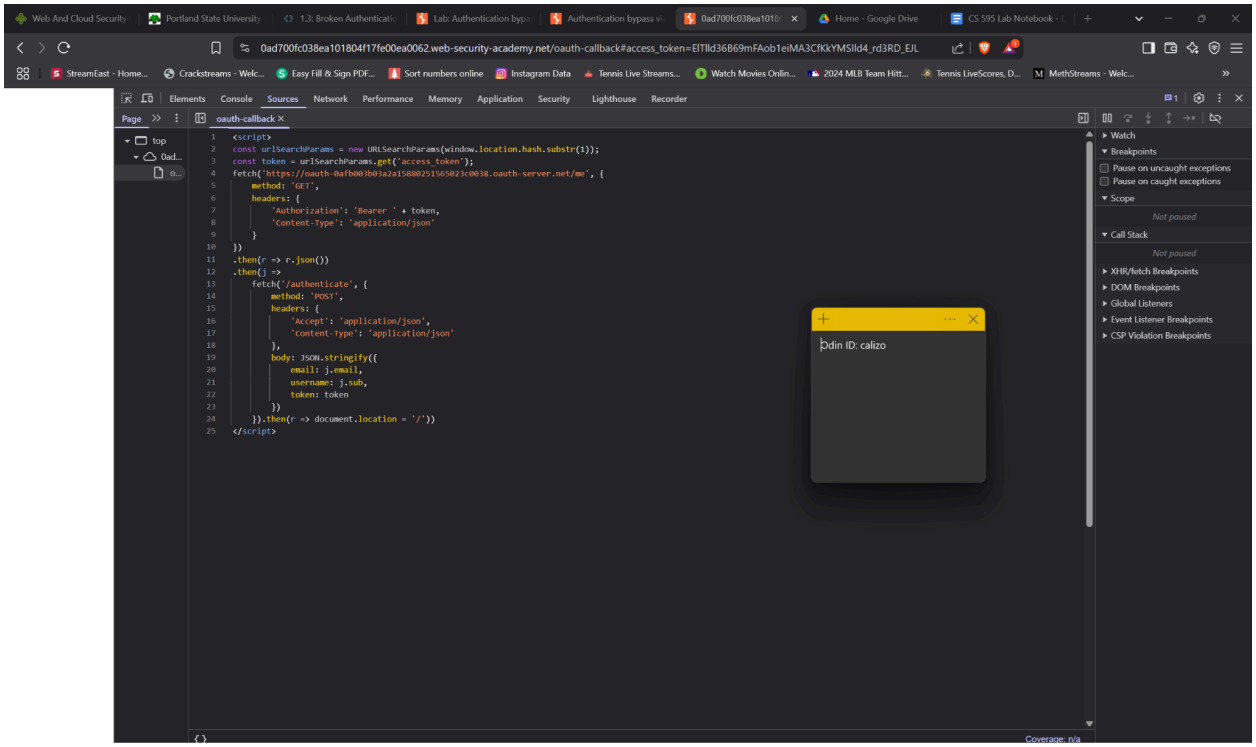
```
    'username' : 'wiener',
    'password' : 'peter'
}
resp = s.post(login_url, data=login_data)
```

After authentication, the server redirects the user to a consent page (granting the client access to profile/email info). The script then captures the Continue button's form action:

```
soup = BeautifulSoup(resp.text,'html.parser')
cont = soup.find('form')
cont_url = f"https://{oauth_site}{cont['action']}"
print(f'Continue URL is: {cont_url}')
```

OAuth Callback Analysis and Implicit Flow

After the OAuth provider redirects the user back to the client application via its /oauth-callback route, I analyzed the behavior of the implicit flow by inspecting the inlined JavaScript source before the redirect could fully resolve. I captured the code using DevTools → Sources tab → oauth-callback, immediately after navigating to the callback URL with a valid access token.



The following observations and breakdowns were made:

```
const urlSearchParams = new URLSearchParams(window.location.hash.substr(1));
const token = urlSearchParams.get('access_token');
```

- These two lines extract the access token (access_token=...) from the hash fragment of the redirected URL. This value was previously issued by the identity provider after the successful OAuth login and user consent.

```
fetch('https://oauth-0afb003b03a2a15880251565023c0038.oauth-server.net/me',
{
  method: 'GET',
  headers: {
    'Authorization': 'Bearer ' + token,
    'Content-Type': 'application/json'
  }
})
```

- The script uses the extracted token to make a GET request to the identity provider's /me endpoint. This fetch is meant to return the user profile (e.g., email, sub) associated with the token. This part is still secure and expected behavior in an OAuth implementation.

```
.then(r => r.json())
.then(j =>
  fetch('/authenticate', {
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      email: j.email,
      username: j.sub,
      token: token
    })
  }).then(r => document.location = '/')
)
```

Here’s where the vulnerability kicks in. Instead of the client application verifying the identity server-side (e.g., by calling /me itself), it relies on the user's browser to pass their identity via the /authenticate endpoint. This includes:

- email: retrieved from the previous fetch response
- username: the user's subject ID
- token: the access token

Because this logic resides entirely on the client-side and trusts the data fetched by the browser, a malicious user could tamper with the email field in the POST /authenticate payload and impersonate a different user. The flow then ends with a redirect to /, logging in the user under the assumed identity.

This lab illustrated the dangers of trusting client-side identity propagation in an OAuth implicit flow. The vulnerability stems from lack of server-side token introspection. In this case, the client application should've independently verified the access token against the identity provider’s /me endpoint instead of letting the browser dictate the user’s identity. I now see how this kind of oversight can lead to full account takeover with minimal friction.

OAuth Exploitation and Final Account Takeover

With the access token successfully harvested in Step 10, this final phase demonstrates how a vulnerable OAuth implementation can be hijacked through token reuse and improper identity validation. The core flaw lies in the fact that the client application relies on the user to fetch their own identity data from the identity provider, then self-report that data to /authenticate. This breaks the implicit trust boundary that OAuth is designed to enforce.

To start, I issued a direct GET request to the identity provider's /me endpoint to simulate what the JavaScript on the callback page does. The token is passed in the Authorization header as a Bearer token. The identity provider validated the token and returned the correct data for the wiener account:


```
{
  "sub": "wiener",
  "name": "Peter Wiener",
  "email": "wiener@hotdog.com",
  "email_verified": true
}
```

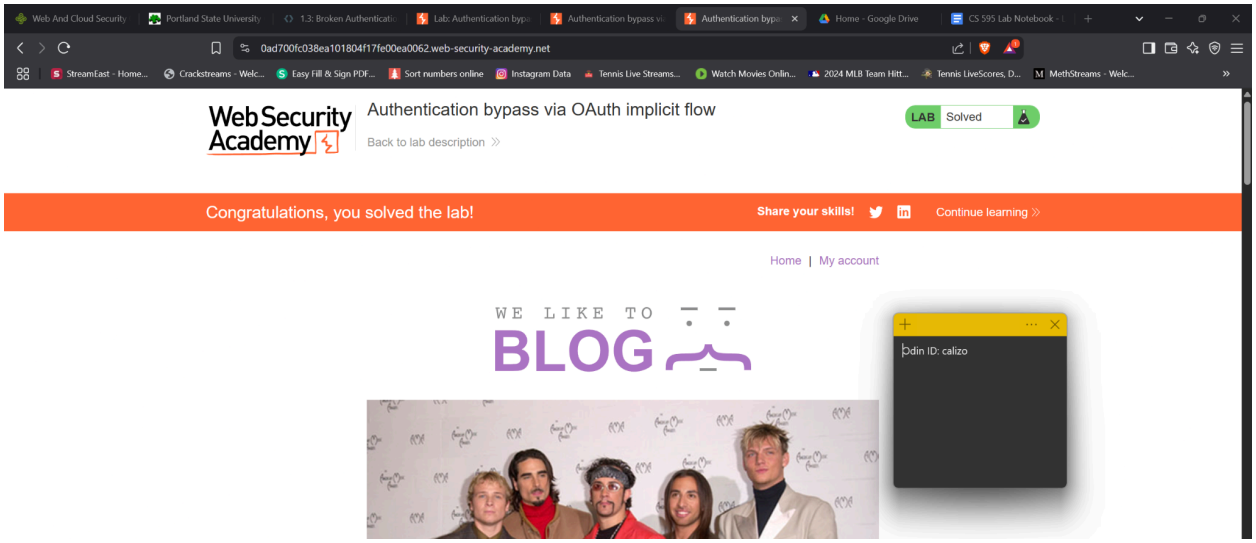

This proves the token was issued to wiener, not carlos. Rather than submit this legitimate data back to the client, I forged the POST request to /authenticate, reusing the valid token but spoofing the email and username fields to appear as carlos:

```
authenticate_data = {
    'email': 'carlos@carlos-montoya.net',
    'username': 'carlos',
    'token': token  # still wiener's token
```

}

This worked—because the client never validated the email from /me on the server side, it blindly trusted the POST body sent by the browser.

After submitting this crafted request, I visited the /my-account?id=carlos endpoint and confirmed successful access to Carlos's account. The final lab screen verified this with a "Congratulations, you solved the lab!" message and my Odin ID (calizo) visibly displayed alongside.



This lab reinforces a core lesson in OAuth security: never trust identity claims from the client—always verify token content on the server using a secure and direct channel. Relying on user-submitted data from token-based authentication breaks the OAuth threat model and invites identity spoofing attacks like this one.